

ĐẠI HỌC BÁCH KHOA HÀ NỘI

TÀI LIỆU ÔN BẢO VỆ ĐỒ ÁN

Bộ phân tích mạch khuếch đại

Kiến thức nền — Công thức — Giải thích source code

Biên soạn từ source và báo cáo hiện tại của dự án

Contents

1	Câu giới thiệu đề tài trong 30 giây	1
I	PHẦN 1 — KIẾN THỨC NỀN	2
1	Bài toán thực sự của thiết bị	2
1.1	Vì sao phải đánh giá từng thông số?	2
2	Tín hiệu sine và các đại lượng cơ bản	3
2.1	Vì sao dùng sine?	3
3	Tần số: đo để làm gì?	4
4	Biên độ, Vpp, RMS và offset	4
4.1	Vmax, Vmin, Vpp	4
4.2	Giá trị trung bình	4
4.3	RMS toàn phần và RMS AC	4
5	Gain của DUT	5
5.1	Ba loại “gain” dễ nhầm	5
6	Pha và độ trễ	5
7	Đồ thị Bode	6
7.1	Vì sao trục tần số logarithm?	6
7.2	Đọc Bode như thế nào?	6
8	Lấy mẫu, Nyquist và số mẫu trên chu kỳ	7
9	Aliasing	7
10	DAC bậc thang, ZOH và settling time	7
11	ADC 12 bit, lượng tử hóa và saturation	8
11.1	Saturation và clipping khác nhau	8
12	Range và AFE	8
13	Calibration	8
14	Nhiều và sine-fit residual	9
15	PASS, FAIL và WARNING	9
II	PHẦN 2 — GIẢI THÍCH CÔNG THỨC TRONG BÁO CÁO	10
1	Mean và RMS AC	10
2	Gain tuyến tính và gain dB	10

3 Sine fit	11
4 Pha và chuẩn hóa góc	11
5 Độ trễ	11
6 Số mẫu trên chu kỳ	11
7 Chu kỳ và khoảng lấy mẫu	12
8 Sai số bỏ lỡ đỉnh	12
9 ZOH droop	12
10 Settling margin	12
11 Đổi mã ADC sang điện áp — điểm phải trả lời cẩn thận	13
12 Gain của DUT không đảo	13
13 Khôi phục DC CH2	14
14 CRC/XOR của frame	14
 III PHẦN 3 — GIẢI THÍCH SOURCE CODE	 15
1 Nên đọc source theo thứ tự nào?	15
2 Firmware: vai trò từng module	15
3 main.c: chương trình bắt đầu ở đâu?	16
4 Các build mode	16
5 command_parser.c: giao thức điều khiển	17
6 test_controller.c: cấu hình phép đo và DAC	17
6.1 Tạo LUT	17
6.2 test_controller_configure()	18
6.3 START	18
7 mcp4822.c: DAC hoạt động thế nào?	18
8 ads7861.c: ADC hai kênh	19
9 adc_stream.c: phần khó nhất của firmware	19
9.1 Mục tiêu	19
9.2 Luồng thời gian	19
9.3 Ring buffer	19
9.4 Tại sao dùng sequence number?	20
9.5 Frame stream hiện tại	20
9.6 Vì sao đạt khoảng 140 kSPS thay vì luôn 200 kSPS?	20

10	measurement_engine.c	20
11	range_control.c	21
12	calibration.c	21
13	protocol.c và USB CDC	21
14	Desktop: cấu trúc tổng thể	21
15	stream_reader_process.py: tại sao là process riêng?	22
16	signal_analysis.py: DSP chính	22
16.1	calculate_sampling_quality()	22
16.2	raw_adc_to_volts() và convert_measurement_channels()	22
16.3	_positive_zero_crossing_frequency()	22
16.4	_sine_fit()	23
16.5	analyze_channel()	23
16.6	analyze_dut()	23
16.7	evaluate_pass_fail()	23
17	signal_analyzer.py: GUI và worker	23
17.1	SignalAnalyzerApp	23
17.2	LiveStreamWorker	23
17.3	Xử lý một block	23
17.4	Downsampling hiển thị	24
18	Bode worker	24
19	Unit test chứng minh điều gì?	24
20	Luồng end-to-end nên trình bày khi bảo vệ	25
20.1	Đo live	25
20.2	Quét Bode	25
IV	CÂU HỎI HỘI ĐỒNG DỄ HỎI	26
1	Tại sao không dùng ADC/DAC nội STM32?	26
2	Tại sao chọn STM32F103?	26
3	Tại sao phải lấy mẫu đồng thời?	26
4	Tại sao không chỉ dùng V_{pp} để tính gain?	26
5	20 kHz có đo được không?	26
6	Vì sao gain Bode đúng mà pha cao tần lệch?	26
7	Tại sao range không phải gain DUT?	27
8	Vì sao có FAIL dù gain nằm trong dung sai?	27

9	CRC hiện tại có mạnh không?	27
10	Đóng góp chính của đề án là gì?	27
11	Hạn chế lớn nhất?	27
12	Nếu có thêm thời gian sẽ làm gì trước?	27
V	PHẦN ÔN NHANH TRƯỚC KHI VÀO BẢO VỆ	28
1	Kịch bản trình bày 2 phút	28

Tài liệu này giải thích đúng theo phiên bản source hiện tại của dự án. Mục tiêu là giúp hiểu bản chất để tự trả lời, không phải học thuộc từng dòng code.

1 Câu giới thiệu đề tài trong 30 giây

Đề tài thiết kế một bộ phân tích mạch khuếch đại. Thiết bị dùng DAC MCP4822 tạo tín hiệu kích thích, đưa tín hiệu qua DUT, dùng ADC lấy mẫu đồng thời ADS7861 đo Vin và Vout, STM32F103 điều khiển thời gian và truyền dữ liệu qua USB CDC. Ứng dụng desktop khôi phục điện áp, hiển thị dạng sóng, tính tần số, biên độ, gain, pha và quét Bode. Điểm chính của đề tài không chỉ là “đo ra một con số”, mà còn đánh giá điều kiện đo để phân biệt PASS, FAIL và WARNING.

PHẦN 1 — KIẾN THỨC NỀN

1 Bài toán thực sự của thiết bị

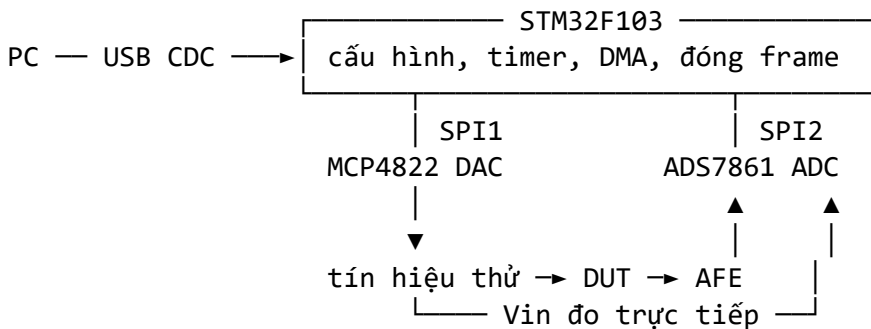
Một bộ khuếch đại có đầu vào và đầu ra:

Nguồn phát → Vin → DUT → Vout

Muốn đánh giá DUT, cần biết:

- DUT có giữ đúng tần số không?
- Điện áp đầu ra lớn hơn đầu vào bao nhiêu lần?
- Đầu ra sớm hay trễ pha so với đầu vào?
- Gain và pha có thay đổi theo tần số không?
- Tín hiệu có bị méo, clipping hoặc bão hòa đường đo không?
- Điều kiện lấy mẫu có đủ tốt để tin vào kết quả không?

Kiến trúc của đề tài:



ADS7861 lấy mẫu hai kênh gần như đồng thời. Điều này quan trọng khi đo pha: nếu hai kênh được đo ở hai thời điểm khác nhau thì bản thân độ lệch thời gian của ADC sẽ biến thành sai số pha.

1.1 Vì sao phải đánh giá từng thông số?

Thông số	Câu hỏi kỹ thuật mà nó trả lời
Tần số	Nguồn phát có đúng không, DUT có giữ đúng thành phần cơ bản không?
Biên độ/Vpp	Tín hiệu lớn đến đâu, có sử dụng đúng range và có chạm giới hạn không?
Mean/offset	Điểm làm việc DC có đúng không?

Thông số	Câu hỏi kỹ thuật mà nó trả lời
RMS AC	Năng lượng phần tín hiệu biến thiên là bao nhiêu, không bị offset làm sai?
Gain	DUT khuếch đại hoặc suy hao bao nhiêu?
Pha/delay	DUT làm tín hiệu sớm/trễ bao nhiêu, có ảnh hưởng ổn định và timing không?
Noise residual	Bao nhiêu thành phần không được mô hình sine giải thích?
Clipping/saturation	Kết quả còn thuộc vùng tuyến tính và miền đo hợp lệ không?
Bode	Gain và pha thay đổi thế nào theo tần số; bảng thông hữu ích nằm ở đâu?
Sampling quality	Các con số trên có đủ điều kiện để tin cậy hay chỉ mang tính tham khảo?

2 Tín hiệu sine và các đại lượng cơ bản

Một tín hiệu sine được viết:

$$v(t) = A \sin(2\pi ft + \phi) + C$$

Trong đó:

- A: biên độ đỉnh, đơn vị V.
- f: tần số, đơn vị Hz.
- ϕ : pha ban đầu, đơn vị độ hoặc radian.
- C: thành phần DC/offset.
- Chu kỳ $T = 1/f$.

Ví dụ tín hiệu $2 \pm 1 \text{ V}$, 200 Hz nghĩa là:

- Offset: 2 V.
- Biên độ đỉnh: 1 V.
- Vmax lý tưởng: 3 V.
- Vmin lý tưởng: 1 V.
- Vpp: 2 V.
- Chu kỳ: 5 ms.

2.1 Vì sao dùng sine?

Sine là tín hiệu cơ bản của hệ tuyến tính. Nếu đưa một sine qua hệ tuyến tính bất biến theo thời gian, đầu ra vẫn là sine cùng tần số; chỉ thay đổi biên độ và pha. Vì vậy chỉ cần so sánh Vin/Vout là có thể suy ra đáp ứng của DUT tại tần số đó.

Square và triangle chứa nhiều thành phần hài nên hữu ích khi quan sát đáp ứng quá độ hoặc méo, nhưng khó quy toàn bộ kết quả về một gain và một pha duy nhất.

3 Tần số: đo để làm gì?

Tần số cho biết số chu kỳ trong một giây. Thiết bị kiểm tra tần số vì:

- Xác nhận DAC phát đúng cấu hình.
- Xác nhận DUT không tạo dao động bất thường hoặc làm mất dạng tín hiệu.
- Là tham số bắt buộc để tính chu kỳ, pha, độ trễ và số mẫu/chu kỳ.
- Là trục ngang của đồ thị Bode.

Desktop ước lượng tần số bằng các lần tín hiệu đi qua mức trung bình theo chiều dương. Nó nội suy vị trí zero crossing giữa hai mẫu để giảm sai số lượng tử thời gian, sau đó lấy trung vị các chu kỳ.

Điểm yếu: zero crossing nhảy với nhiễu. Nếu tín hiệu rất nhỏ, nhiễu có thể tạo thêm nhiều giao điểm giả.

4 Biên độ, Vpp, RMS và offset

4.1 Vmax, Vmin, Vpp

$$V_{pp} = V_{max} - V_{min}$$

Vpp trực quan và dễ kiểm tra trên oscilloscope. Tuy nhiên, nếu không lấy mẫu đúng đỉnh thì Vmax bị thấp và Vpp bị đánh giá thiếu.

4.2 Giá trị trung bình

Giá trị trung bình biểu diễn thành phần DC. Với tín hiệu 2 ± 1 V, mean lý tưởng là 2 V.

4.3 RMS toàn phần và RMS AC

- RMS toàn phần tính cả DC và AC.
- RMS AC trừ mean trước, chỉ phản ánh năng lượng biến thiên.

Với sine biên độ đỉnh A:

$$V_{RMS,AC} = \frac{A}{\sqrt{2}}$$

Tại sao dự án dùng RMS AC để tính gain? Vì Vin và Vout có thể có offset khác nhau do bias và mạch AFE. Nếu dùng RMS toàn phần, offset sẽ làm sai gain. RMS AC cũng ít phụ thuộc vào việc có lấy trùng đỉnh hay không.

5 Gain của DUT

Gain tuyến tính:

$$G = \frac{V_{out}}{V_{in}}$$

Trong dự án, tử và mẫu là RMS AC của hai kênh. Nếu DUT gain 3 thì biên độ AC đầu ra lý tưởng gấp ba đầu vào.

Gain dB:

$$G_{dB} = 20 \log_{10}(G)$$

Các mốc cần nhớ:

Gain tuyến tính	Gain dB
0,5	−6,02 dB
0,8	−1,94 dB
1	0 dB
2	6,02 dB
3	9,54 dB
10	20 dB

Vì sao điện áp dùng hệ số 20 chứ không phải 10? Công suất tỷ lệ với bình phương điện áp. Từ $10 \log_{10}(P_{out}/P_{in})$ và cùng trở kháng suy ra $20 \log_{10}(V_{out}/V_{in})$.

5.1 Ba loại “gain” dễ nhầm

1. **Gain DUT**: đại lượng cần đo, ví dụ 3 lần.
2. **Bit gain X1/X2 của MCP4822**: lựa chọn thang điện áp của DAC, không phải gain DUT.
3. **Hệ số range x10/x100 của đường CH2**: hệ số điều hòa tín hiệu trước ADC; app phải nhân/chia bù để khôi phục Vout thật, cũng không phải gain DUT.

Đây là câu rất dễ bị hỏi.

6 Pha và độ trễ

Hai sine cùng tần số có thể lệch nhau một góc $\Delta\phi$.

- $\Delta\phi > 0$: theo quy ước của code, Vout sớm pha so với Vin.
- $\Delta\phi < 0$: Vout trễ pha.
- Kết quả được chuẩn hóa về $[-180^\circ, 180^\circ]$ để tránh các biểu diễn tương đương như 350° và -10° .

Độ trễ tương đương:

$$\tau = \frac{\Delta\phi}{360f}$$

Ví dụ $f = 1$ kHz, lệch pha 36° tương đương:

$$\tau = \frac{36}{360 \times 1000} = 100 \mu s$$

Tại sao đo pha? Gain đúng chưa đủ. Một mạch có thể khuếch đại đúng biên độ nhưng gây trễ lớn, ảnh hưởng ổn định vòng điều khiển, băng thông và chất lượng tín hiệu.

Lưu ý của nguyên mẫu: pha Bode ở tần số cao còn gồm trễ của DAC, AFE, ADC, đường truyền và thuật toán. Vì vậy phải hiệu chuẩn pha theo tần số trước khi công bố như một thiết bị đo chuẩn.

7 Đồ thị Bode

Đáp ứng tần số của DUT là:

$$H(j\omega) = \frac{V_{out}(j\omega)}{V_{in}(j\omega)}$$

Đồ thị Bode gồm:

- Đồ thị magnitude: $20 \log_{10} |H|$ theo trục tần số logarithm.
- Đồ thị phase: $\arg(H)$ theo trục tần số logarithm.

7.1 Vì sao trục tần số logarithm?

Một thiết bị thường hoạt động qua nhiều bậc tần số, ví dụ 100 Hz đến 20 kHz. Trục log cho phép nhìn đồng thời vùng thấp và cao, đồng thời phù hợp với cách mô tả octave/decade.

7.2 Đọc Bode như thế nào?

- Vùng gain gần như phẳng: passband.
- Tần số gain giảm 3 dB so với passband thường được xem là tần số cắt của hệ bậc một.
- Sau tần số cắt, hệ bậc một lý tưởng giảm khoảng 20 dB/decade.
- Pha thay đổi cho biết trễ động học của hệ.

Trong kết quả hiện tại, gain phần cứng khoảng 9,26–9,47 dB, gần gain mục tiêu 3 lần là 9,54 dB. Đường pha tăng sai lệch rõ ở vùng cao nên chỉ nên nói: **chức năng quét Bode đã chạy, đường gain có ý nghĩa quan sát; pha cần hiệu chuẩn trước khi dùng định lượng.**

8 Lấy mẫu, Nyquist và số mẫu trên chu kỳ

Nyquist yêu cầu:

$$F_s > 2f_{max}$$

Điều này chỉ đảm bảo về lý thuyết có thể tránh aliasing khi tín hiệu đã được giới hạn băng thông. Nó không đảm bảo waveform vẽ đẹp hoặc đo peak/phase chính xác.

Số mẫu mỗi chu kỳ:

$$N = \frac{F_s}{f}$$

Với $F_s = 140077$ SPS:

Tần số	Mẫu/chu kỳ xấp xỉ	Ý nghĩa
100 Hz	1401	Dạng sóng rất mịn
1 kHz	140	Tốt
5 kHz	28	Có thể phân tích, ít điểm hơn
10 kHz	14	Chỉ phù hợp nguyên mẫu
20 kHz	7	Đủ nhận diện tần số nhưng yếu cho peak/pha

Nếu hội đồng hỏi “20 kHz có thỏa Nyquist không?”, trả lời: **có, vì 140 kSPS lớn hơn 40 kSPS; nhưng chỉ có khoảng 7 mẫu/chu kỳ nên độ tin cậy của peak, hình dạng và pha vẫn thấp.**

9 Aliasing

Aliasing xảy ra khi thành phần trên $F_s/2$ bị ánh xạ xuống tần số thấp. Khi đã alias, phần mềm không thể biết đó là tần số thật hay ảnh giả chỉ từ chuỗi mẫu.

Cách hạn chế:

- Chọn F_s đủ cao.
- Dùng lọc chống alias trước ADC.
- Giới hạn băng thông tín hiệu đầu vào.

10 DAC bậc thang, ZOH và settling time

DAC chỉ cập nhật tại các thời điểm rời rạc rồi giữ giá trị đến lần cập nhật tiếp theo. Đây là zero-order hold (ZOH), tạo dạng bậc thang và suy hao biên độ ở tần số cao.

MCP4822 còn cần khoảng 4,5 μs để đầu ra xác lập. Nếu cập nhật mỗi 5 μs thì biên thời gian chỉ còn 0,5 μs . Nghĩa là hệ thống vẫn chạy nhưng rất sát giới hạn.

Phân biệt:

- **DAC update rate:** số điểm DAC xuất mỗi giây.
- **Tần số sine:** số chu kỳ sine mỗi giây.
- $N_{DAC} = \text{update_rate} / f$ là số điểm DAC trên một chu kỳ.

11 ADC 12 bit, lượng tử hóa và saturation

ADC 12 bit có 4096 mã. Độ phân giải lý tưởng của ADS7861 theo điện áp vi sai là xấp xỉ:

$$LSB = \frac{V_{REF}}{2048}$$

vì dữ liệu gốc là signed 12-bit với miền $-2048 \dots 2047$.

Lượng tử hóa làm điện áp bị làm tròn theo LSB. Sai số tương đối trở nên đáng kể khi tín hiệu nhỏ.

11.1 Saturation và clipping khác nhau

- **ADC saturation:** mã ADC chạm gần 0 hoặc 4095; đường đo đã hết range.
- **Signal clipping:** waveform có đoạn đỉnh/đáy bị bẹt. Nó có thể xảy ra ở DUT, op-amp AFE hoặc ADC.
- Saturation thường gây clipping, nhưng clipping có thể xuất hiện trước ADC mà mã chưa chạm đúng rail.

12 Range và AFE

Vout đi qua op-amp đảo ghép AC và relay chọn điện trở hồi tiếp. Ba nhánh giúp đưa các mức Vout khác nhau về miền ADC.

Hệ số khôi phục mặc định trong app:

range 0,3 V: $-10/47$
 range 3,3 V: $-10/4,7$
 range 10 V: $-10/1,5$

Dấu âm bù cho op-amp đảo. Đây là hệ số của đường đo CH2, không phải gain DUT.

Relay dùng break-before-make: tắt tất cả relay, chờ, sau đó bật relay mới. Mục đích là tránh hai nhánh hồi tiếp cùng đóng.

13 Calibration

Calibration tuyến tính dùng:

$$V_{true} = mV_{raw} + c$$

- m: bù sai số gain.

- **c**: bù offset.

Mỗi range CH2 có cặp hệ số riêng. CH1 là đường trực tiếp nên chỉ dùng calibration CH1 range 0. Calibration tuyến tính không bù được hoàn toàn:

- Sai số thay đổi theo tần số.
- Trễ pha.
- Méo phi tuyến.
- Jitter.
- Clipping.

14 Nhiễu và sine-fit residual

Sau khi khớp sine, phần dư là:

$$e[n] = v[n] - v_{fit}[n]$$

RMS của phần dư được app gọi là `noise_rms`. Nó gồm nhiễu, hài, sai tần số mô hình và mọi thành phần không được mô hình sine giải thích. Vì vậy không nên khẳng định đây là nhiễu thuần túy hoặc THD+N chuẩn phòng thí nghiệm.

15 PASS, FAIL và WARNING

- **PASS**: spec đạt, dữ liệu đầy đủ, không clipping/saturation và điều kiện đo đủ tin cậy.
- **FAIL**: ít nhất một yêu cầu kỹ thuật không đạt, mất dữ liệu hoặc lỗi truyền thông.
- **WARNING**: chưa thấy vi phạm spec chính nhưng điều kiện đo hạn chế độ tin cậy.

Điểm thiết kế hay của dự án: gain đúng không đồng nghĩa kết quả tổng phải PASS. Ví dụ 20 kHz/200 kSPS có 10 mẫu/chu kỳ, gain có thể đúng nhưng vẫn WARNING.

PHẦN 2 — GIẢI THÍCH CÔNG THỨC TRONG BÁO CÁO

1 Mean và RMS AC

$$\bar{v} = \frac{1}{M} \sum_{n=0}^{M-1} v[n]$$

M là số mẫu. Mean là mức DC trung bình.

$$V_{RMS,AC} = \sqrt{\frac{1}{M} \sum_{n=0}^{M-1} (v[n] - \bar{v})^2}$$

Các bước code thực hiện:

1. Tính mean.
2. Trừ mean khỏi từng mẫu.
3. Bình phương.
4. Lấy trung bình.
5. Lấy căn.

Ví dụ $v(t) = 0,3 \sin(\omega t) + 0,1$: mean = 0,1 V; RMS AC = $0,3/\sqrt{2} \approx 0,2121$ V.

2 Gain tuyến tính và gain dB

$$G_{lin} = \frac{V_{out,RMS,AC}}{V_{in,RMS,AC}}$$

Nếu $V_{out\ RMS} = 0,4243$ V và $V_{in\ RMS} = 0,2121$ V thì gain = 2.

$$G_{dB} = 20 \log_{10}(G_{lin})$$

Gain error:

$$E_G = G_{measured,dB} - G_{target,dB}$$

Đạt nếu $|E_G| \leq \text{tolerance}$.

3 Sine fit

Báo cáo viết:

$$v(t) = A \sin(2\pi ft + \phi) + C + e(t)$$

Code không giải phi tuyến trực tiếp. Nó biến đổi:

$$v(t) = a \sin(2\pi ft) + b \cos(2\pi ft) + C$$

rồi dùng least squares tìm **a**, **b**, **C**. Sau đó:

$$A = \sqrt{a^2 + b^2}, \quad \phi = \text{atan2}(b, a)$$

Đây là lý do code tạo ma trận ba cột **sin**, **cos**, **1** trong **_sine_fit()**.

Ưu điểm: tận dụng toàn bộ mẫu và ước lượng amplitude/phase tốt hơn chỉ lấy max/min. Nhược điểm: cần biết tần số fit tương đối đúng và tín hiệu phải gần sine.

4 Pha và chuẩn hóa góc

$$\Delta\phi = \phi_{out} - \phi_{in}$$

Code chuẩn hóa:

`(phase + 180) % 360 - 180`

Ví dụ 350° trở thành -10° , vì hai giá trị mô tả cùng một độ lệch vật lý.

Phase error cũng được chuẩn hóa theo cách tương tự so với phase mục tiêu.

5 Độ trễ

$$\tau = \frac{\Delta\phi}{360f}$$

Nếu cần μs thì nhân 10^6 .

Lưu ý: đây là độ trễ tương đương tại một tần số, không nhất thiết bằng group delay trên toàn dải.

6 Số mẫu trên chu kỳ

$$N = \frac{F_s}{f}$$

Ví dụ $F_s=140077$, $f=20$ kHz:

$$N \approx 7,00$$

Pha cách nhau giữa hai mẫu là:

$$\Delta\theta = \frac{360^\circ}{N}$$

Với $N = 7$, mỗi mẫu cách nhau hơn 51° , nên vị trí đỉnh và pha rất nhạy.

7 Chu kỳ và khoảng lấy mẫu

$$T_{\text{signal}} = \frac{1}{f}, \quad T_s = \frac{1}{F_s}$$

Báo cáo đổi sang μs bằng cách dùng 10^6 ở tử.

8 Sai số bỏ lỡ đỉnh

Trường hợp xấu nhất, mẫu gần đỉnh bị lệch nửa bước lấy mẫu. Khoảng pha đó là π/N , nên biên độ mẫu được chỉ còn $A \cos(\pi/N)$.

$$E_{\text{peak}} = \left(1 - \cos \frac{\pi}{N}\right) 100\%$$

Với $N = 10$:

$$E_{\text{peak}} \approx 4,894\%$$

Đây là sai số xấu nhất của cách đo peak bằng mẫu rời rạc, không phải sai số bắt buộc của sine fit.

9 ZOH droop

Đáp ứng biên độ của zero-order hold có dạng sinc. Tại tần số tín hiệu:

$$E_{\text{ZOH}} = \left(1 - \frac{\sin(\pi/N)}{\pi/N}\right) 100\%$$

N càng lớn, suy hao càng nhỏ. Công thức này đánh giá lý tưởng của quá trình giữ mẫu, chưa bao gồm settling, sai số DAC và mạch lọc.

10 Settling margin

$$T_{\text{margin}} = T_s - T_{\text{settling}}$$

Với $F_s = 200 \text{ kSPS}$: $T_s = 5 \mu\text{s}$; MCP4822 settling khoảng $4,5 \mu\text{s}$; margin = $0,5 \mu\text{s}$ nên code báo borderline.

Nếu $F_s = 500 \text{ kSPS}$: $T_s = 2 \mu\text{s}$, margin = $-2,5 \mu\text{s}$; DAC chưa kịp ổn định trước lần cập nhật tiếp theo.

11 Đổi mã ADC sang điện áp — điểm phải trả lời cẩn thận

Báo cáo đang có công thức đơn giản hóa theo miền 3,3 V:

$$V = m_r \left(\frac{code}{4095} 3,3 - 1,65 \right) + c_r$$

Nhưng **source hiện tại của ADS7861 không dùng đúng công thức này**. Source dùng dữ liệu signed 12-bit, chuyển sang offset-binary để truyền:

$$raw_{signed} = code - 2048$$

$$V_{shifted} = \frac{raw_{signed}}{2048} V_{REF}, \quad V_{REF} = 2,5V$$

sau đó:

$$V = mV_{shifted} + c$$

Trong desktop:

- CH1 còn cộng thêm 2,5 V vì đây là đường Vin trực tiếp có bias theo cấu hình hiện tại.
- CH2 áp hệ số range của AFE.

Nếu bị hỏi, nên trả lời: “Công thức 3,3 V trong báo cáo là biểu diễn tổng quát/đơn giản hóa. Pipeline chạy thật bám ADS7861 VREF 2,5 V và midpoint 2048 như trong `calibration_adc_code_to_voltage()` và `raw_adc_to_volts()`.” Sau bảo vệ nên sửa báo cáo để hai phần hoàn toàn thống nhất.

12 Gain của DUT không đảo

$$A_v = 1 + \frac{R_f}{R_g}$$

Với $R_f = 20 \text{ k}\Omega$, $R_g = 10 \text{ k}\Omega$:

$$A_v = 3$$

$$G_{dB} = 20 \log_{10}(3) \approx 9,54dB$$

Công thức chỉ đúng khi op-amp đầu không đảo, có hồi tiếp âm, chưa clipping và còn đủ gain-bandwidth/output swing.

13 Khôi phục DC CH2

AFE CH2 ghép AC nên không đo trực tiếp mức DC thật của Vout. Khi bật tùy chọn khôi phục, app giả sử DUT tuyến tính đi qua gốc:

$$V_{out} = GV_{in}$$

App suy ra độ lớn gain từ năng lượng AC, lấy dấu từ correlation rồi ước lượng:

$$V_{out,DC} \approx G_{signed}V_{in,DC}$$

Đây là **giá trị suy ra theo mô hình**, không phải DC đo trực tiếp. Không nên dùng nó để công bố đặc tính DC của một DUT có offset riêng.

14 CRC/XOR của frame

Source gọi hàm là CRC nhưng phép tính hiện tại là XOR tất cả byte:

$$checksum = b_0 \oplus b_1 \oplus \dots \oplus b_{L-1}$$

Ưu điểm: rất nhẹ. Nhược điểm: yếu hơn CRC-16/CRC-32; một số tổ hợp nhiều lỗi bit có thể triệt tiêu nhau. Sequence number mới là cơ chế phát hiện mất cả block.

PHẦN 3 — GIẢI THÍCH SOURCE CODE

1 Nên đọc source theo thứ tự nào?

Không đọc từ trên xuống theo thư mục. Hãy đọc theo luồng dữ liệu:

1. `amplifier_analyzer_fw/Core/Src/main.c`
2. `command_parser.c`
3. `test_controller.c`
4. `mcp4822.c` và `ads7861.c`
5. `adc_stream.c`
6. `protocol.c`
7. `range_control.c` và `calibration.c`
8. `app_desktop/stream_reader_process.py`
9. `app_desktop/signal_analysis.py`
10. `app_desktop/signal_analyzer.py`
11. Các file `test_*.py`

2 Firmware: vai trò từng module

File	Vai trò
<code>main.c</code>	Khởi tạo clock, GPIO, SPI, USB và gọi service trong vòng lặp chính
<code>command_parser.c</code>	Nhận lệnh ASCII, parse tham số, gọi module tương ứng
<code>test_controller.c</code>	Giữ cấu hình phép đo, tạo LUT DAC, start/stop DAC và finite capture
<code>mcp4822.c</code>	Đóng frame 16-bit và điều khiển DAC MCP4822
<code>ads7861.c</code>	Điều khiển mode, conversion, đọc/parse hai word ADC
<code>adc_stream.c</code>	Pipeline timer–SPI–DMA, ring buffer và stream USB liên tục
<code>measurement_engine.c</code>	Tính mean, Vpp, RMS, gain và pha phía firmware
<code>range_control.c</code>	Điều khiển ba relay manual/auto, break-before-make
<code>calibration.c</code>	Hệ số DAC/ADC, đổi code–voltage và lưu Flash

File	Vai trò
protocol.c	Gửi frame nhị phân, checksum XOR, chờ USB TX
usbd_cdc_if.c	Cầu nối USB CDC của STM32 HAL với parser

3 main.c: chương trình bắt đầu ở đâu?

Trình tự khởi tạo:

```
HAL_Init
→ SystemClock_Config (72 MHz)
→ GPIO
→ SPI1 cho DAC
→ SPI2 cho ADC
→ USB re-enumeration
→ USB CDC
→ test_controller_init
→ command_parser_init
→ mcp4822_init
→ ads7861_init + self-test parse
→ adc_stream_init
```

Vòng lặp chính production chỉ chạy các service nhẹ:

```
command_parser_process();
adc_stream_usb_service();
test_controller_service();
```

Ý nghĩa kiến trúc:

- Việc đúng thời gian nằm trong timer/DMA interrupt.
- Main loop xử lý command và đóng/gửi USB.
- Không thực hiện phép đọc blocking dài trong ISR.

4 Các build mode

config.h/PlatformIO cho phép tách lỗi:

- Production/NORMAL: chạy toàn hệ thống.
- USB SIM: không cần analog, sinh dữ liệu giả để kiểm tra protocol/app.
- MCP4822 test: chỉ kiểm tra DAC.
- Các mode ADC, SPI loopback, calibration Flash: phục vụ bring-up.

Lợi ích: nếu app không nhận dữ liệu, có thể phân biệt lỗi USB, DAC, ADC hay chuỗi analog thay vì debug toàn hệ thống cùng lúc.

5 command_parser.c: giao thức điều khiển

USB callback không thực thi toàn bộ command; nó gom dòng lệnh. Main loop gọi parser để tránh làm việc nặng trong interrupt USB.

Các lệnh quan trọng:

Lệnh	Chức năng
PING, INFO	Kiểm tra kết nối và firmware
CONFIG: . . .	Cấu hình waveform, f, amplitude, offset, DAC gain, Fs, samples
START, STOP	Khởi động/dừng DAC và phép đo
GET_RESULT	Nhận kết quả JSON phía firmware
GET_SAMPLES	Nhận finite frame nhị phân
ADC_USB_STREAM_START: FB	Bắt đầu ADC stream liên tục
ADC_STREAM_STOP/STATUSD	Dừng/đọc thống kê stream
SET_RANGE, GET_RANGE	Chọn relay range
GET_CALIB, SET_CALIB, SAVE_CALIB	Quản lý hiệu chuẩn

Các lệnh ADC_GPIO_DIAG, ADC_READ_ONCE, trace... là lệnh chẩn đoán bring-up, không phải luồng đo bình thường.

6 test_controller.c: cấu hình phép đo và DAC

current_config giữ:

- Dạng sóng.
- Tần số.
- Biên độ và offset.
- DAC gain X1/X2.
- Fs.
- Số mẫu finite capture.

6.1 Tạo LUT

test_controller_generate_lut() chọn số điểm:

$N = \text{DAC_MAX_DMA_RATE} / \text{frequency}$
giới hạn $4 \leq N \leq 256$

Sau đó tính sine/square/triangle/DC, cộng bias 1,65 V và chuyển điện áp sang mã DAC qua calibration.

DAC stream dùng TIM3 + DMA SPI1. Một LUT đúng bằng một chu kỳ; timer phát tuần hoàn LUT nên tần số thật là $\text{update_rate}/N$.

6.2 test_controller_configure()

Hàm kiểm tra:

- Samples hữu hạn không vượt 256.
- Tần số hợp lệ và không vượt khả năng LUT/DAC.
- Fs không nhỏ hơn tần số.
- Gain DAC hợp lệ.
- Biên độ + offset không vượt miền unipolar của DAC.

6.3 START

- Đảm bảo DAC đang phát.
- Trong finite mode, đọc một block ADC.
- Chạy auto-range nếu cần và thu lại sau khi đổi relay.
- Gọi measurement engine.

Live stream của app sau đó dùng pipeline `adc_stream.c`, không lặp `GET_SAMPLES` liên tục.

7 mcp4822.c: DAC hoạt động thế nào?

MCP4822 nhận word 16-bit gồm:

- Chọn kênh A/B.
- Bit gain.
- Shutdown/active.
- 12 bit dữ liệu.

Driver làm ba việc chính:

1. `mcp4822_build_frame()` tạo word đúng bitfield.
2. Hàm write giữ CS thấp trong cả 16 clock SPI.
3. DMA path phát liên tục mà không bắt CPU ghi từng mẫu.

LDAC dùng để chốt đầu ra. Bit gain của MCP4822 chỉ đổi thang DAC, hoàn toàn độc lập với gain DUT.

8 ads7861.c: ADC hai kênh

ADS7861 xuất word có status và dữ liệu signed 12-bit. Driver:

- Cấu hình M0/M1 và chọn cặp A0/B0.
- Tạo xung RD/CONVST.
- Đọc tuần tự word A và B qua Serial Data A.
- Parse status, bỏ các bit không phải dữ liệu.
- Sign-extend dữ liệu 12-bit.
- Kiểm tra frame hợp lệ.

Theo wiring hiện tại:

- ADS B0 là Vin.
- ADS A0 là Vout.

Khi đóng gói cho app, firmware đổi two's-complement thành offset-binary để code 2048 tương ứng 0 V vì sai.

9 adc_stream.c: phần khó nhất của firmware

9.1 Mục tiêu

Thu hai kênh đều theo timer mà không để USB hoặc GUI làm chậm nhịp ADC.

9.2 Luồng thời gian

TIM2 update
→ phát xung conversion
→ SPI2 RX/TX DMA đọc word A
→ half-transfer interrupt chuyển sang word B
→ hoàn thành cặp A/B
→ lưu ring buffer
→ main loop đóng frame USB khi đủ 512 cặp

9.3 Ring buffer

- Kích thước: 2048 cặp mẫu.
- Chunk USB: 512 cặp.
- Producer: ISR ADC.
- Consumer: main loop USB service.

Nếu producer bắt kịp consumer, không mất mẫu. Nếu ring đầy, code bỏ nguyên chunk cũ và tăng `ring_overwrite`; sequence sẽ nhảy để host phát hiện.

9.4 Tại sao dùng sequence number?

CRC chỉ biết frame nhận được có lỗi byte hay không. Sequence biết giữa hai frame có bị mất cả block hay không.

9.5 Frame stream hiện tại

```
AA BB | type 04 | payload length
sequence: 4 byte
actual Fs: 4 byte
count: 2 byte
data: 512 × 3 byte
XOR: 1 byte
```

Hai kênh 12 bit được pack thành 24 bit = 3 byte/cặp, giảm từ 4 byte/cặp xuống 3 byte/cặp.

9.6 Vì sao đạt khoảng 140 kSPS thay vì luôn 200 kSPS?

Giới hạn thực tế gồm thời gian SPI ADC, ngắt timer/DMA, đóng frame, USB FS, RAM nhỏ và khả năng host drain COM. 140 kSPS là mức đã kiểm chứng ổn định end-to-end với GUI hiện tại; 200 kSPS từng có nguy cơ sequence gap.

10 measurement_engine.c

Firmware tính:

- Mean, min, max, Vpp.
- RMS AC.
- Gain dB.
- Pha bằng cross-correlation theo lag nguyên.

Pha firmware bị lượng tử theo một mẫu:

$$\Delta\phi_{step} = 360 \frac{f}{F_s} = \frac{360}{N}$$

Vì vậy desktop tính lại pha bằng sine fit trên dữ liệu thô để có độ phân giải tốt hơn. Nếu bị hỏi “tại sao tính hai lần?”, trả lời: firmware cho kết quả nhanh/độc lập; desktop có tài nguyên lớn hơn và dùng thuật toán ổn định hơn cho hiển thị chính.

11 range_control.c

Manual mode bật range do người dùng chọn. Auto mode quan sát khoảng cách mã ADC tới midpoint và số mẫu gần rail:

- Gần rail → tăng range.
- Tín hiệu quá nhỏ → giảm range.
- Sau khi đổi relay → chờ xác lập và thu lại.

Break-before-make tránh đóng hai relay đồng thời.

12 calibration.c

`calibration_reset()` đặt hệ số mặc định. `calibration_save()` ghi struct vào trang Flash cuối.

Hai hàm quan trọng:

- `calibration_voltage_to_dac_code()`: điện áp mong muốn → mã DAC.
- `calibration_adc_code_to_voltage()`: mã offset-binary → điện áp đã bù range.

Hạn chế hiện tại: dữ liệu Flash chưa có magic/version/CRC mạnh. Code mới chỉ kiểm tra trang có trắng `0xFFFFFFFF` hay không.

13 protocol.c và USB CDC

`protocol_send_raw_async()` cố gửi frame mà không block quá lâu. Double buffer trong stream cho phép một frame đang truyền trong khi frame kia được xây dựng.

Tên `protocol_calculate_crc()` dễ gây hiểu nhầm: thuật toán là XOR 1 byte, không phải CRC đa thức.

14 Desktop: cấu trúc tổng thể

File	Vai trò
<code>signal_analyzer.py</code>	GUI, serial control, worker, plot, export
<code>signal_analysis.py</code>	Toàn bộ DSP và logic PASS/FAIL/WARNING có thể test độc lập
<code>stream_reader_process.py</code>	Tiến trình riêng sở hữu COM và đọc stream tốc độ cao
<code>test_signal_analysis.py</code>	Unit test công thức, range và DSP
<code>test_bode_sweep.py</code>	Test chọn Fs Bode và decode frame packed 12-bit
<code>_capture_report_tests.py</code>	Tự động chạy cấu hình và chụp ảnh cho báo cáo

15 stream_reader_process.py: tại sao là process riêng?

Nếu GUI vừa render plot vừa đọc COM trong cùng process, Python GIL và callback GUI có thể khiến COM không được drain kịp. Windows tạo back-pressure về STM32 và làm mất block.

Giải pháp:

1. Process con mở COM độc quyền.
2. Gửi START và ADC_USB_STREAM_START.
3. Đọc, kiểm header, length, XOR và sequence.
4. Decode packed 12-bit.
5. Gom nhiều frame thành block 4096 mẫu.
6. Gửi block về GUI qua socket localhost bằng message có length prefix.

Nhờ vậy việc vẽ chậm không trực tiếp chặn reader COM.

16 signal_analysis.py: DSP chính

16.1 calculate_sampling_quality()

Tính T, Ts, N, peak error, ZOH droop, settling margin và tạo warning.

Ngưỡng hiện tại:

- $N < 10$: sampling too low.
- $10 \leq N < 25$: POC only.
- Peak error $> 1\%$: cảnh báo.
- Settling margin < 0 : DAC not settled.
- Margin từ 0 đến dưới $1 \mu s$: borderline.

16.2 raw_adc_to_volts() và convert_measurement_channels()

- Trừ midpoint 2048.
- Nhân VREF 2,5 V.
- CH1 dùng calibration trực tiếp và cộng bias.
- CH2 dùng hệ số range.

16.3 _positive_zero_crossing_frequency()

- Trừ mean.

- Tìm cặp mẫu đi từ âm/0 sang dương.
- Nội suy vị trí crossing.
- Lấy median chu kỳ.

16.4 `_sine_fit()`

Least squares trên sin/cos/constant để tìm amplitude, phase, offset và residual.

16.5 `analyze_channel()`

Tạo toàn bộ metric một kênh. Saturation dựa trên raw code gần rail. Clipping dựa trên chuỗi ít nhất năm mẫu liên tiếp gần đỉnh hoặc đáy; điều kiện span giúp tránh coi nhiễu lượng tử của tín hiệu gần phẳng là clipping.

16.6 `analyze_dut()`

- $\text{Gain} = \text{RMSout}/\text{RMSin}$.
- Gain error so với target.
- $\text{Phase} = \text{phase CH2} - \text{phase CH1}$.
- Phase error so với target.
- Tính delay.

16.7 `evaluate_pass_fail()`

Ưu tiên failure. Nếu không có failure nhưng có sampling warning thì WARNING. Chỉ khi không có cả hai mới PASS.

17 `signal_analyzer.py`: GUI và worker

17.1 `SignalAnalyzerApp`

Quản lý widget, serial, cấu hình DUT, calibration, plot, history, export và trạng thái.

17.2 `LiveStreamWorker`

Khởi động process reader, nhận block qua socket rồi phát Qt signal về GUI.

17.3 Xử lý một block

```
raw code  
→ convert channels  
→ tùy chọn khôi phục DC CH2
```

- `analyze_channel(CH1/CH2)`
- `analyze_dut`
- `calculate_sampling_quality`
- `evaluate_pass_fail`
- cập nhật waveform và bảng kết quả

17.4 Downsampling hiển thị

App có thể nhận rất nhiều mẫu nhưng không vẽ tất cả. Downsampling chỉ phục vụ render; DSP vẫn dùng block dữ liệu thật. Các index được giữ theo timestamp thật để không tạo ramp giả.

18 Bode worker

Mỗi điểm Bode phản ứng:

1. Dừng stream cũ và DAC.
2. Gửi CONFIG với `SAMPLES=256`.
3. START DAC.
4. Chờ 5 chu kỳ, tối thiểu 20 ms và tối đa 300 ms.
5. Start ADC USB stream với $F_s = \min(\max(50f, 5 \text{ kSPS}), 140 \text{ kSPS})$.
6. Bỏ frame đầu sau chuyển trạng thái.
7. Thu 1024 mẫu, kiểm CRC và sequence.
8. Đổi code sang điện áp.
9. Sine fit hai kênh, tính gain và pha.
10. Chỉ vẽ điểm hợp lệ; không dùng sentinel -99 dB.

Không cần thay firmware vì app tái sử dụng CONFIG, START và ADC_USB_STREAM_START hiện có.

19 Unit test chứng minh điều gì?

`test_signal_analysis.py` kiểm tra:

- Các case N/peak/ZOH/settling.
- RMS AC loại offset.
- Gain 2 và pha -30° .
- Gain/phase DUT độc lập với range.
- Warning không bị biến thành PASS.
- Đổi mã ADS7861.

- Range chỉ tác động CH2.
- Downsampling giữ timestamp.

test_bode_sweep.py kiểm tra:

- Fs Bode được chọn và cap đúng 140 kSPS.
- Decode đúng frame packed 12-bit cho cả hai kênh.

Unit test chứng minh thuật toán với dữ liệu xác định trước, không thay thế phép hiệu chuẩn phần cứng.

20 Luồng end-to-end nên trình bày khi bảo vệ

20.1 Đo live

Người dùng cấu hình app

- USB CONFIG
- STM32 tạo LUT và chạy DAC DMA
- DUT khuếch đại
- AFE chọn range
- TIM2 kích ADS7861
- SPI2 DMA thu A/B
- ring buffer
- packed USB frame + sequence + XOR
- process desktop đọc COM
- đổi code sang volt
- DSP
- waveform + DUT table + PASS/FAIL/WARNING

20.2 Quét Bode

dãy tần số log

- lập cấu hình từng f
- chờ xác lập
- thu block
- sine fit Vin/Vout
- gain dB + phase
- vẽ hai đường theo trục log

CÂU HỎI HỘI ĐỒNG DỄ HỎI

1 Tại sao không dùng ADC/DAC nội STM32?

MCP4822 có DAC 12-bit, tham chiếu nội và giao tiếp SPI rõ ràng; ADS7861 có hai ADC lấy mẫu đồng thời, phù hợp đo pha. Việc tách ngoại vi cũng phản ánh đúng bài toán thiết kế thiết bị đo và cho phép kiểm soát timing/range tốt hơn.

2 Tại sao chọn STM32F103?

Đủ cho nguyên mẫu: 72 MHz, SPI, timer, DMA và USB FS, phổ biến và chi phí thấp. Hạn chế là RAM 20 kB và biên xử lý/USB nhỏ khi muốn tăng lên 500 kSPS, nên hướng nâng cấp là STM32H7 hoặc nền tảng mạnh hơn.

3 Tại sao phải lấy mẫu đồng thời?

Vì nếu Vin và Vout đo lệch thời điểm, khoảng lệch đó tạo pha giả $360f\Delta t$. ADC đồng thời giảm sai số này.

4 Tại sao không chỉ dùng Vpp để tính gain?

Vpp nhạy với việc bỏ lỡ đỉnh, nhiễu và clipping. RMS AC dùng toàn bộ mẫu, loại offset và ổn định hơn. Sine fit còn ước lượng amplitude/phase từ toàn bộ block.

5 20 kHz có đo được không?

Có thể nhận diện tần số và ước lượng gain, vì Fs khoảng 140 kSPS thỏa Nyquist. Tuy nhiên chỉ có 7 mẫu/chu kỳ, nên peak, waveform và pha có độ tin cậy thấp; hệ thống phải cảnh báo và chưa được coi là thiết bị chuẩn tại điểm này.

6 Vì sao gain Bode đúng mà pha cao tần lệch?

Gain từ tỷ số RMS/sine amplitude ít nhạy với trễ cố định. Pha cộng dồn trễ của DAC, AFE, ADC, timing hai đường và thuật toán; pha cần được đo loopback và bù theo tần số.

7 Tại sao range không phải gain DUT?

Range chỉ là hệ số điều hòa Vout để phù hợp miền ADC. App dùng hệ số calibration nghịch để khôi phục Vout thật rồi mới chia cho Vin. Gain DUT là tỷ số sau khôi phục.

8 Vì sao có FAIL dù gain nằm trong dung sai?

Kết quả tổng kiểm tra nhiều tiêu chí: gain, pha, tần số, biên độ, clipping, saturation, communication, data loss và chất lượng lấy mẫu. Một tiêu chí khác fail thì trạng thái tổng vẫn FAIL.

9 CRC hiện tại có mạnh không?

Không. Hiện tại là XOR một byte, nhẹ nhưng khả năng phát hiện lỗi hạn chế. Sequence phát hiện mất block; hướng cải tiến là CRC-16/CRC-32.

10 Đóng góp chính của đề án là gì?

Thiết kế và tích hợp được chuỗi hoàn chỉnh từ phát, DUT, AFE/range, ADC đồng thời, firmware timer-SPI-DMA, USB stream đến ứng dụng phân tích và đánh giá độ tin cậy của phép đo. Kết quả hiện tại là nguyên mẫu chức năng, chưa thay thế thiết bị đo đã hiệu chuẩn.

11 Hạn chế lớn nhất?

- Khoảng 140 kSPS chỉ cho 7 mẫu/chu kỳ ở 20 kHz.
- DAC 200 kupdate/s sát settling 4,5 μ s.
- Pha Bode chưa hiệu chuẩn.
- CH2 DC có phần suy ra, không đo trực tiếp.
- Calibration Flash chưa có version/CRC.
- XOR yếu hơn CRC chuẩn.

12 Nếu có thêm thời gian sẽ làm gì trước?

Ưu tiên hiệu chuẩn loopback gain/phase theo tần số, xác nhận bằng máy phát và oscilloscope chuẩn; sau đó tăng tốc nền tảng ADC/MCU/USB, thêm anti-alias filter và CRC mạnh.

PHẦN ÔN NHANH TRƯỚC KHI VÀO BẢO VỆ

Hãy nhớ 12 ý sau:

1. Đề tài là bộ phân tích DUT, không chỉ là oscilloscope.
2. DAC phát; ADC đồng thời đo Vin/Vout; STM32 điều phối; app phân tích.
3. DUT mục tiêu gain 3 = 9,54 dB.
4. Gain DUT khác DAC X1/X2 và khác range CH2.
5. Gain tính bằng tỷ số RMS AC.
6. Pha desktop tính bằng sine fit.
7. Bode là gain và pha theo tần số log.
8. Nyquist đúng chưa có nghĩa waveform/phase chính xác.
9. 140 kSPS tại 20 kHz chỉ có 7 mẫu/chu kỳ.
10. Streaming dùng timer–SPI–DMA, ring 2048, chunk 512 và sequence.
11. PASS/FAIL/WARNING đánh giá cả spec lẫn độ tin cậy.
12. Nguyên mẫu đã chạy end-to-end nhưng pha cao tần và độ chính xác tuyệt đối vẫn cần hiệu chuẩn.

1 Kịch bản trình bày 2 phút

“Thiết bị tạo tín hiệu bằng MCP4822, đưa qua DUT rồi đo đồng thời Vin và Vout bằng ADS7861. STM32F103 sử dụng timer và DMA để tách timing lấy mẫu khỏi việc truyền USB. Dữ liệu hai kênh được đóng frame có checksum và sequence, sau đó ứng dụng desktop khôi phục điện áp theo calibration/range. Phần mềm dùng RMS AC để tính gain nhằm loại offset, dùng sine fit để tính biên độ và pha, đồng thời đánh giá số mẫu mỗi chu kỳ, sai số bỏ lỡ đỉnh, ZOH và settling DAC. DUT thử nghiệm có gain lý thuyết 3 lần, tương đương 9,54 dB; kết quả Bode gain đo khoảng 9,26 đến 9,47 dB. Hệ thống đã hoạt động end-to-end, nhưng tại 20 kHz tốc độ 140 kSPS chỉ còn 7 mẫu mỗi chu kỳ và pha chưa hiệu chuẩn, nên đây là nguyên mẫu chức năng chứ chưa phải thiết bị đo chuẩn.”